Ανάλυση Δεδομένων 2025–26

4η Εργασία: Μέθοδοι Δέντρων και Συνόλων Ταξινομητών

Πρόβλεψη τύπου κρασιού με Decision Tree, Bagging, Random Forest και Boosting

Ρακοβαλής Παύλος
ΑΕΜ: 6931

27 Απριλίου 2026

Περίληψη

Στην παρούσα εργασία χρησιμοποιούνται τα δεδομένα της 3ης εργασίας με στόχο την πρόβλεψη του τύπου κρασιού, δηλαδή αν ένα κρασί είναι λευκό ή κόκκινο. Ως προβλεπτικές μεταβλητές χρησιμοποιούνται όλα τα φυσικοχημικά χαρακτηριστικά εκτός από τη μεταβλητή `quality`. Εφαρμόζονται τέσσερις μέθοδοι ταξινόμησης: δέντρο ταξινόμησης, Bagging, Random Forest και Boosting. Αρχικά αξιολογούνται τα μοντέλα με τις υπερπαραμέτρους που ζητούνται στην εκφώνηση και στη συνέχεια μελετάται η επίδραση του βάθους του δέντρου και του αριθμού εκτιμητών. Τέλος, πραγματοποιείται βελτιστοποίηση υπερπαραμέτρων με Grid Search και 5-fold stratified cross-validation. Τα αποτελέσματα δείχνουν ότι όλες οι ensemble μέθοδοι έχουν πολύ υψηλή ακρίβεια, ενώ το καλύτερο βελτιστοποιημένο μοντέλο είναι το Bagging με test accuracy 0.9729.

Περιεχόμενα

1 Εισαγωγή και περιγραφή δεδομένων **3**
1.1 Κριτήριο αξιολόγησης 3

2 Ερώτημα 1: Δέντρο ταξινόμησης μέγιστου βάθους 2 **4**
2.1 Αποτελέσματα 4
2.2 Ερμηνεία του δέντρου 5

3 Ερώτημα 2: Bagging με 200 εκτιμητές **5**
3.1 Αποτελέσματα 5
3.2 Σχόλιο 6

4 Ερώτημα 3: Random Forest με 200 εκτιμητές και $m = p/2$ **6**
4.1 Αποτελέσματα 6
4.2 Σχόλιο 7

5 Ερώτημα 4: Boosting με 200 επιμέρους μοντέλα 7
5.1 Αποτελέσματα 7
5.2 Σχόλιο 8

6 Ερώτημα 5: Test error του δέντρου ως συνάρτηση του βάθους 8
6.1 Αριθμητικά αποτελέσματα 8
6.2 Σχολιασμός 9

7 Ερώτημα 6: Test error των ensemble μεθόδων ως συνάρτηση του `n_estimators` 10
7.1 Αριθμητικά αποτελέσματα 10
7.2 Σχολιασμός ανά μέθοδο 11
7.3 Συμπέρασμα ερωτήματος 6 12

8 Ερώτημα 7: Hyperparameter tuning και τελική σύγκριση 12
8.1 Διαδικασία tuning 12
8.2 Πλέγματα υπερπαραμέτρων 13
8.3 Βέλτιστες υπερπαράμετροι και tuned test accuracy 13
8.4 Σύγκριση untuned και tuned μοντέλων 13
8.5 Πίνακες σύγχυσης tuned μοντέλων 14
8.6 Σχολιασμός της επίδρασης του tuning 14
8.7 Καλύτερη μέθοδος ταξινόμησης 14

9 Συνολικά συμπεράσματα 15

Α΄ Αρχεία υλοποίησης 16

1 Εισαγωγή και περιγραφή δεδομένων

Στόχος της 4ης εργασίας είναι η εκτίμηση του τύπου κρασιού (`wine_type`), δηλαδή η ταξινόμηση κάθε παρατήρησης ως λευκό (`white`) ή κόκκινο (`red`) κρασί. Για την ανάλυση χρησιμοποιούνται τα δεδομένα της 3ης εργασίας. Το αρχικό αρχείο περιλαμβάνει $5\,295$ παρατηρήσεις και 13 μεταβλητές: 11 φυσικοχημικά χαρακτηριστικά, τη μεταβλητή `quality` και τη μεταβλητή στόχο `wine_type`.

Σύμφωνα με την εκφώνηση, η μεταβλητή `quality` δεν χρησιμοποιείται ως προβλεπτική μεταβλητή. Άρα το πλήθος των προβλεπτικών μεταβλητών είναι $p = 11$:

`fixed acidity`	`volatile acidity`	`citric acid`
`residual sugar`	`chlorides`	`free sulfur dioxide`
`total sulfur dioxide`	`density`	`pH`
`sulphates`	`alcohol`	

Η μεταβλητή απόκρισης κωδικοποιήθηκε ως:

`red = 0` και `white = 1`.

Η κατανομή των κλάσεων στο σύνολο δεδομένων είναι:

Πίνακας 1: Κατανομή της μεταβλητής `wine_type`.

Κλάση	**Πλήθος**	**Ποσοστό**
`white`	3991	75.37%
`red`	1304	24.63%

Παρατηρείται ότι οι κλάσεις δεν είναι ισορροπημένες, καθώς τα λευκά κρασιά είναι περίπου τριπλάσια από τα κόκκινα. Για τον λόγο αυτό, ο διαχωρισμός σε training και test έγινε με `stratify=y`, ώστε να διατηρηθεί περίπου η ίδια αναλογία κλάσεων και στα δύο υποσύνολα.

Σε όλα τα ερωτήματα χρησιμοποιήθηκε διαχωρισμός:

$$70\% \text{ training set}, \qquad 30\% \text{ test set}.$$

Το `random_state` τέθηκε ίσο με τον ΑΕΜ:

`random_state = 6931`.

Έτσι προέκυψαν:

$$n_{\text{train}} = 3706, \qquad n_{\text{test}} = 1589.$$

Στο test set υπάρχουν 391 κόκκινα και 1198 λευκά κρασιά.

1.1 Κριτήριο αξιολόγησης

Για κάθε μοντέλο υπολογίζεται η ακρίβεια στο test set:

$$\text{Test Accuracy} = \frac{\text{αριθμός σωστών προβλέψεων}}{\text{σύνολο παρατηρήσεων test}}.$$

Επίσης, σύμφωνα με τα ερωτήματα 5 και 6, χρησιμοποιείται το test error:

$$\text{Test Error} = 1 - \text{Test Accuracy}.$$

Οι πίνακες σύγχυσης παρουσιάζονται με γραμμές τις πραγματικές κλάσεις και στήλες τις προβλεπόμενες κλάσεις:

	pred red	**pred white**
true red	TN	FP
true white	FN	TP

2 Ερώτημα 1: Δέντρο ταξινόμησης μέγιστου βάθους 2

Στο πρώτο ερώτημα εφαρμόστηκε δέντρο ταξινόμησης με μέγιστο βάθος 2. Το μοντέλο που χρησιμοποιήθηκε ήταν:

```
DecisionTreeClassifier(max_depth=2, random_state=6931)
```

Το μικρό βάθος περιορίζει την πολυπλοκότητα του μοντέλου. Έτσι το δέντρο είναι εύκολα ερμηνεύσιμο, αλλά πιθανόν να μην μπορεί να αποτυπώσει όλες τις σύνθετες σχέσεις μεταξύ των μεταβλητών και της κλάσης.

2.1 Αποτελέσματα

Πίνακας 2: Αποτελέσματα δέντρου ταξινόμησης με `max_depth=2`.

Μοντέλο	**max depth**	**Test Accuracy**	**Test Error**	**Λάθη**
Decision Tree	2	0.9339	0.0661	105

Πίνακας 3: Πίνακας σύγχυσης για το δέντρο ταξινόμησης.

	pred red	**pred white**
true red	360	31
true white	74	1124

Το μοντέλο ταξινόμησε σωστά $360 + 1124 = 1484$ από τις 1589 παρατηρήσεις του test set. Η τελική ακρίβεια είναι:

$$\frac{1484}{1589} = 0.9339.$$

2.2 Ερμηνεία του δέντρου

Οι κανόνες που προέκυψαν από το δέντρο είναι:

```
total sulfur dioxide <= 67.50
|-- chlorides <= 0.05  -> white
|-- chlorides >  0.05  -> red

total sulfur dioxide > 67.50
|-- chlorides <= 0.07  -> white
|-- chlorides >  0.07  -> red
```

Άρα, για βάθος 2, το δέντρο βασίζεται κυρίως σε δύο μεταβλητές: `total sulfur dioxide` και `chlorides`. Αυτό είναι λογικό για το συγκεκριμένο πρόβλημα, καθώς τα λευκά και τα κόκκινα κρασιά συχνά διαφέρουν αρκετά στις τιμές των θειωδών και των χλωριδίων. Παρά την απλότητά του, το μοντέλο πετυχαίνει πολύ καλή ακρίβεια, περίπου 93.39%.

3 Ερώτημα 2: Bagging με 200 εκτιμητές

Στο δεύτερο ερώτημα εφαρμόστηκε η μέθοδος Bagging (Bootstrap Aggregating). Η βασική ιδέα της μεθόδου είναι ότι εκπαιδεύονται πολλά δέντρα ταξινόμησης σε διαφορετικά bootstrap δείγματα του training set και η τελική πρόβλεψη προκύπτει με πλειοψηφική ψηφοφορία.

Το μοντέλο που χρησιμοποιήθηκε ήταν:

```
BaggingClassifier(estimator=DecisionTreeClassifier(),
        n_estimators=200, random_state=6931)
```

Σε αντίθεση με το απλό δέντρο του ερωτήματος 1, εδώ δεν χρησιμοποιείται μόνο ένα δέντρο, αλλά σύνολο 200 δέντρων. Αυτό μειώνει τη διακύμανση της πρόβλεψης, καθώς τα λάθη μεμονωμένων δέντρων τείνουν να εξομαλύνονται μέσω της ψηφοφορίας.

3.1 Αποτελέσματα

Πίνακας 4: Αποτελέσματα Bagging με 200 εκτιμητές.

Μοντέλο	**n estimators**	**Test Accuracy**	**Test Error**	**Λάθη**
Bagging	200	0.9660	0.0340	54

Πίνακας 5: Πίνακας σύγχυσης για το Bagging.

	pred red	**pred white**
true red	365	26
true white	28	1170

Το Bagging ταξινόμησε σωστά $365+1170 = 1535$ από τις 1589 παρατηρήσεις. Η ακρίβεια αυξήθηκε από 0.9339 στο απλό δέντρο σε 0.9660, ενώ το test error μειώθηκε από 0.0661 σε 0.0340.

3.2 Σχόλιο

Η βελτίωση σε σχέση με το απλό δέντρο είναι σημαντική. Το απλό δέντρο περιορίζεται σε λίγους κανόνες, ενώ το Bagging συνδυάζει πολλά δέντρα και μειώνει την αστάθεια που χαρακτηρίζει τα δέντρα απόφασης. Επομένως, η μέθοδος πετυχαίνει καλύτερη γενίκευση στο test set.

4 Ερώτημα 3: Random Forest με 200 εκτιμητές και $m = p/2$

Στο τρίτο ερώτημα εφαρμόστηκε Random Forest. Η μέθοδος είναι επέκταση του Bagging, με τη διαφορά ότι σε κάθε split κάθε δέντρου δεν εξετάζονται όλες οι μεταβλητές, αλλά ένα τυχαίο υποσύνολό τους. Αυτό μειώνει τη συσχέτιση μεταξύ των δέντρων και συχνά οδηγεί σε καλύτερη απόδοση.

Στην παρούσα εργασία το πλήθος των προβλεπτικών μεταβλητών είναι:

$$p = 11.$$

Η εκφώνηση ζητά τυχαίο πλήθος υποψήφιων χαρακτηριστικών:

$$m = \frac{p}{2} = 5.5.$$

Επειδή στο `scikit-learn` το `max_features` πρέπει να είναι ακέραιος αριθμός όταν δηλώνεται ως πλήθος μεταβλητών, χρησιμοποιήθηκε:

$$m = 5.$$

Το μοντέλο ήταν:

```
RandomForestClassifier(n_estimators=200, max_features=5,
                       random_state=6931)
```

4.1 Αποτελέσματα

Πίνακας 6: Αποτελέσματα Random Forest.

Μοντέλο	**n estimators**	**p**	**m**	**Test Accuracy**	**Test Error**
Random Forest	200	11	5	0.9666	0.0334

Πίνακας 7: Πίνακας σύγχυσης για το Random Forest.

	pred red	**pred white**
true red	365	26
true white	27	1171

Το Random Forest ταξινόμησε σωστά $365 + 1171 = 1536$ παρατηρήσεις και έκανε 53 λάθη. Η ακρίβεια ήταν 0.9666, δηλαδή ελαφρώς υψηλότερη από εκείνη του Bagging.

4.2 Σχόλιο

Η διαφορά Random Forest και Bagging είναι μικρή στο συγκεκριμένο πρόβλημα. Ωστόσο, το Random Forest έχει ένα θεωρητικό πλεονέκτημα: επειδή κάθε split εξετάζει μόνο τυχαίο υποσύνολο μεταβλητών, τα δέντρα του ensemble είναι λιγότερο συσχετισμένα μεταξύ τους. Αυτό μπορεί να μειώσει περαιτέρω τη διακύμανση του τελικού μοντέλου. Στα συγκεκριμένα δεδομένα, η βελτίωση είναι οριακή αλλά υπαρκτή.

5 Ερώτημα 4: Boosting με 200 επιμέρους μοντέλα

Στο τέταρτο ερώτημα εφαρμόστηκε Boosting με 200 επιμέρους μοντέλα, συντελεστή μάθησης 0.1 και μέγιστο βάθος 1. Χρησιμοποιήθηκε ο ταξινομητής `GradientBoostingClassifier`, ο οποίος κατασκευάζει διαδοχικά ασθενή μοντέλα, όπου κάθε νέο μοντέλο προσπαθεί να διορθώσει τα λάθη των προηγούμενων.

Το μοντέλο ήταν:

```
GradientBoostingClassifier(n_estimators=200, learning_rate=0.1,
                max_depth=1, random_state=6931)
```

Το `max_depth=1` σημαίνει ότι κάθε επιμέρους μοντέλο είναι δέντρο βάθους 1, δηλαδή decision stump. Κάθε τέτοιο δέντρο είναι πολύ απλό, αλλά ο συνδυασμός πολλών τέτοιων μοντέλων μπορεί να οδηγήσει σε ισχυρό ταξινομητή.

5.1 Αποτελέσματα

Πίνακας 8: Αποτελέσματα Boosting.

Μοντέλο	**n estimators**	**learning rate**	**max depth**	**Test Accuracy**
Boosting	200	0.1	1	0.9679

Πίνακας 9: Πίνακας σύγχυσης για το Boosting.

	pred red	**pred white**
true red	361	30
true white	21	1177

Το Boosting ταξινόμησε σωστά $361 + 1177 = 1538$ παρατηρήσεις και έκανε 51 λάθη. Η ακρίβεια του μοντέλου ήταν 0.9679 και το test error 0.0321.

5.2 Σχόλιο

Μεταξύ των αρχικών μοντέλων των ερωτημάτων 1–4, το Boosting έχει την υψηλότερη ακρίβεια. Η μέθοδος φαίνεται να αξιοποιεί αποτελεσματικά τους ασθενείς ταξινομητές βάθους 1, καθώς κάθε επόμενος ταξινομητής βελτιώνει σταδιακά τις προηγούμενες προβλέψεις. Ιδιαίτερα αξιοσημείωτο είναι ότι το Boosting έκανε μόνο 21 λάθος ταξινομήσεις λευκών κρασιών ως κόκκινα, δηλαδή λιγότερες από τις αντίστοιχες μεθόδους Bagging και Random Forest.

6 Ερώτημα 5: Test error του δέντρου ως συνάρτηση του βάθους

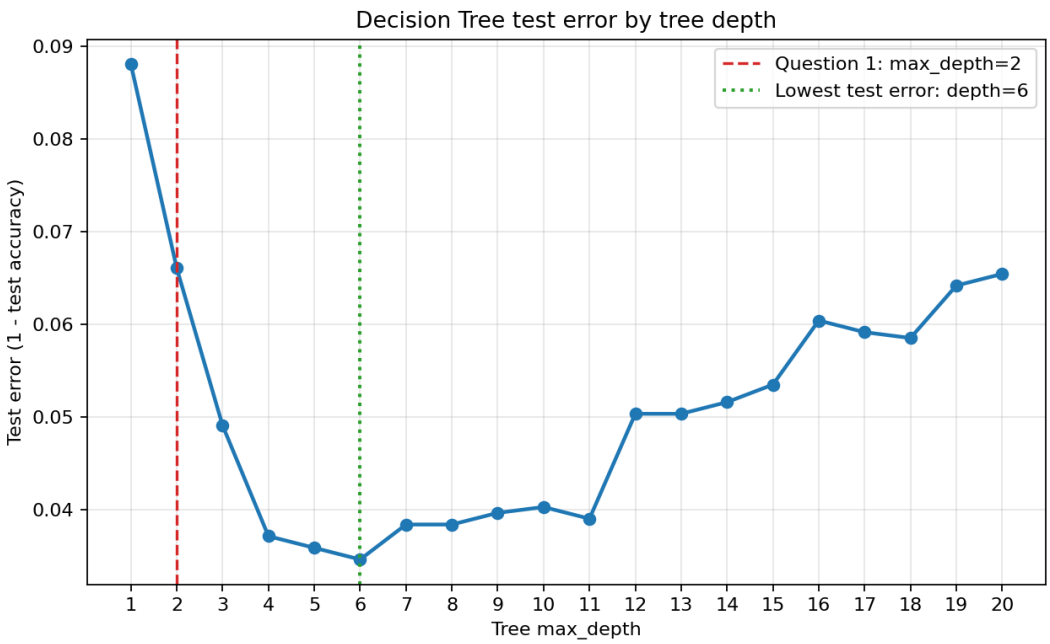
Στο πέμπτο ερώτημα μελετήθηκε η επίδραση του μέγιστου βάθους του δέντρου ταξινόμησης στο test error. Για τον σκοπό αυτό εκπαιδεύτηκαν δέντρα με `max_depth` από 1 έως 20. Για κάθε τιμή του βάθους υπολογίστηκε η ακρίβεια στο ίδιο test set και στη συνέχεια το test error:

$$\text{Test Error} = 1 - \text{Test Accuracy}.$$

6.1 Αριθμητικά αποτελέσματα

Πίνακας 10: Test accuracy και test error για διαφορετικά βάθη δέντρου.

Depth	**Accuracy**	**Error**	**Depth**	**Accuracy**	**Error**
1	0.9119	0.0881	11	0.9610	0.0390
2	0.9339	0.0661	12	0.9497	0.0503
3	0.9509	0.0491	13	0.9497	0.0503
4	0.9629	0.0371	14	0.9484	0.0516
5	0.9641	0.0359	15	0.9465	0.0535
6	0.9654	0.0346	16	0.9396	0.0604
7	0.9616	0.0384	17	0.9408	0.0592
8	0.9616	0.0384	18	0.9415	0.0585
9	0.9604	0.0396	19	0.9358	0.0642
10	0.9597	0.0403	20	0.9346	0.0654



Σχήμα 1: Test error του δέντρου ταξινόμησης συναρτήσει του μέγιστου βάθους.

6.2 Σχολιασμός

Για πολύ μικρά βάθη, το δέντρο είναι υπερβολικά απλό. Για παράδειγμα, στο `max_depth=1` το test error είναι 0.0881, ενώ στο `max_depth=2` μειώνεται σε 0.0661. Αυτό δείχνει ότι το πολύ ρηχό δέντρο δεν έχει αρκετή ευελιξία ώστε να διαχωρίσει ικανοποιητικά τις δύο κλάσεις.

Καθώς το βάθος αυξάνεται από 1 έως 6, το test error μειώνεται συνεχώς. Η καλύτερη τιμή εμφανίζεται στο `max_depth=6`, όπου:

$$\text{Test Accuracy} = 0.9654, \qquad \text{Test Error} = 0.0346.$$

Μετά το βάθος 6, το test error δεν συνεχίζει να μειώνεται. Αντίθετα, παρουσιάζει γενικά ανοδική τάση, ιδιαίτερα για βάθη μεγαλύτερα από 11. Αυτό είναι ένδειξη πιθανής υπερπροσαρμογής: το δέντρο γίνεται πολύ σύνθετο, προσαρμόζεται περισσότερο στις ιδιαιτερότητες του training set και χάνει μέρος της ικανότητας γενίκευσης στο test set.

Συνεπώς, ένα μέτριο βάθος δέντρου είναι καταλληλότερο από ένα πολύ ρηχό ή ένα πολύ βαθύ δέντρο. Το αποτέλεσμα αυτό συμφωνεί και με το tuning του ερωτήματος 7, όπου το βέλτιστο βάθος για το δέντρο ταξινόμησης επιλέχθηκε επίσης ίσο με 6.

7 Ερώτημα 6: Test error των ensemble μεθόδων ως συνάρτηση του `n_estimators`

Στο έκτο ερώτημα μελετήθηκε η επίδραση του αριθμού των εκτιμητών στο test error των τριών ensemble μεθόδων:

- Bagging,
- Random Forest,
- Boosting.

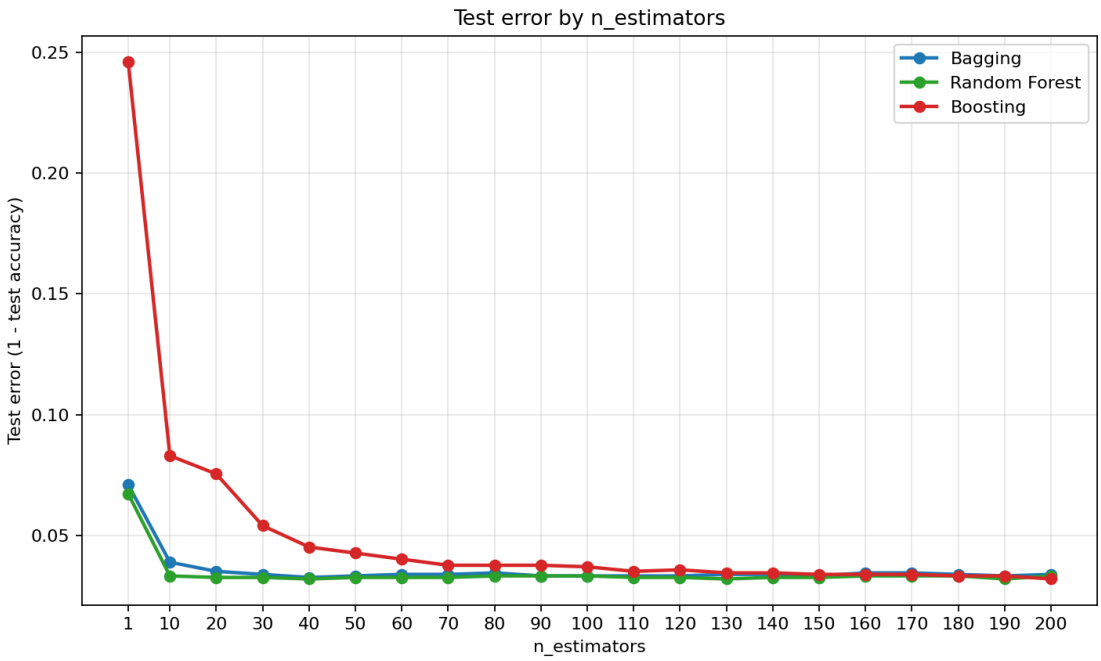
Οι τιμές που εξετάστηκαν ήταν `n_estimators=1, 10, 20, ..., 200`.

Για κάθε μέθοδο και για κάθε τιμή του `n_estimators` εκπαιδεύτηκε νέο μοντέλο στο ίδιο training set και υπολογίστηκε το test error στο ίδιο test set.

7.1 Αριθμητικά αποτελέσματα

Πίνακας 11: Test error των Bagging, Random Forest και Boosting για διαφορετικά `n_estimators`.

n estimators	Bagging	Random Forest	Boosting
1	0.0711	0.0673	0.2461
10	0.0390	0.0334	0.0831
20	0.0352	0.0327	0.0755
30	0.0340	0.0327	0.0541
40	0.0327	0.0321	0.0453
50	0.0334	0.0327	0.0428
60	0.0340	0.0327	0.0403
70	0.0340	0.0327	0.0378
80	0.0346	0.0334	0.0378
90	0.0334	0.0334	0.0378
100	0.0334	0.0334	0.0371
110	0.0334	0.0327	0.0352
120	0.0334	0.0327	0.0359
130	0.0340	0.0321	0.0346
140	0.0340	0.0327	0.0346
150	0.0334	0.0327	0.0340
160	0.0346	0.0334	0.0340
170	0.0346	0.0334	0.0340
180	0.0340	0.0334	0.0334
190	0.0334	0.0321	0.0334
200	0.0340	0.0334	0.0321



Σχήμα 2: Test error των Bagging, Random Forest και Boosting συναρτήσει του αριθμού εκτιμητών.

7.2 Σχολιασμός ανά μέθοδο

Bagging

Για το Bagging, το test error μειώνεται από 0.0711 όταν υπάρχει μόνο ένας εκτιμητής σε 0.0390 για 10 εκτιμητές και σε 0.0352 για 20 εκτιμητές. Μετά από περίπου 40 εκτιμητές, η καμπύλη σταθεροποιείται κοντά στο 0.033–0.034. Το χαμηλότερο test error που παρατηρήθηκε ήταν 0.0327 για `n_estimators=40`.

Άρα η μεγάλη βελτίωση εμφανίζεται στην αρχή, όταν περνάμε από ένα δέντρο σε ένα μικρό ensemble. Η περαιτέρω αύξηση των εκτιμητών προσφέρει μικρή ή μηδενική βελτίωση.

Random Forest

Το Random Forest φτάνει πολύ γρήγορα σε χαμηλό test error. Ήδη για `n_estimators=10` το test error είναι 0.0334, δηλαδή σχεδόν ίδιο με τις μεγαλύτερες τιμές του αριθμού εκτιμητών. Το χαμηλότερο test error ήταν:

$$0.0321,$$

και εμφανίστηκε για `n_estimators=40`, `130` και `190`.

Η καμπύλη του Random Forest είναι η πιο σταθερή από τις τρεις. Αυτό δείχνει ότι η τυχαιοποίηση στα χαρακτηριστικά, σε συνδυασμό με το ensemble δέντρων, οδηγεί σε πολύ καλή απόδοση ακόμη και με σχετικά μικρό αριθμό δέντρων.

Boosting

Το Boosting έχει διαφορετική συμπεριφορά. Για `n_estimators=1`, το test error είναι πολύ υψηλό, ίσο με 0.2461. Αυτό είναι αναμενόμενο, επειδή κάθε επιμέρους μοντέλο είναι πολύ απλό δέντρο βάθους 1. Καθώς αυξάνεται ο αριθμός των εκτιμητών, το test error μειώνεται σταδιακά:

$$0.0831 \text{ για } n = 10, \qquad 0.0453 \text{ για } n = 40, \qquad 0.0321 \text{ για } n = 200.$$

Σε αντίθεση με το Bagging και το Random Forest, το Boosting συνεχίζει να βελτιώνεται μέχρι τις μεγαλύτερες τιμές του `n_estimators`. Αυτό οφείλεται στη σειριακή φύση της μεθόδου: κάθε νέο ασθενές μοντέλο προστίθεται ώστε να διορθώσει σταδιακά τα σφάλματα του προηγούμενου συνόλου μοντέλων.

7.3 Συμπέρασμα ερωτήματος 6

Συνολικά, οι τρεις μέθοδοι καταλήγουν σε πολύ κοντινή απόδοση για μεγάλες τιμές του `n_estimators`. Το Bagging και το Random Forest σταθεροποιούνται πιο γρήγορα, ενώ το Boosting χρειάζεται περισσότερους εκτιμητές για να φτάσει στο ίδιο επίπεδο απόδοσης. Για `n_estimators=200`, το χαμηλότερο test error επιτυγχάνεται από το Boosting:

$$\text{Boosting test error} = 0.0321.$$

8 Ερώτημα 7: Hyperparameter tuning και τελική σύγκριση

Στο έβδομο ερώτημα πραγματοποιήθηκε βελτιστοποίηση υπερπαραμέτρων για κάθε μία από τις τέσσερις μεθόδους. Η βελτιστοποίηση έγινε μόνο στο training set, με Grid Search και 5-fold stratified cross-validation.

Η χρήση stratified cross-validation είναι σημαντική επειδή οι δύο κλάσεις δεν είναι ισορροπημένες. Με αυτόν τον τρόπο κάθε fold διατηρεί περίπου την ίδια αναλογία κόκκινων και λευκών κρασιών.

8.1 Διαδικασία tuning

Για κάθε μοντέλο ορίστηκε πλέγμα υποψήφιων τιμών υπερπαραμέτρων. Για κάθε συνδυασμό υπερπαραμέτρων υπολογίστηκε η μέση ακρίβεια cross-validation. Στη συνέχεια επιλέχθηκε ο συνδυασμός με τη μεγαλύτερη μέση CV accuracy. Το βέλτιστο μοντέλο επανεκπαιδεύτηκε στο training set και αξιολογήθηκε μία φορά στο test set.

Συνοπτικά, η διαδικασία ήταν:

1. Διαχωρισμός δεδομένων σε training/test με `random_state=6931`.
2. Ορισμός πλέγματος υπερπαραμέτρων για κάθε μέθοδο.
3. Εφαρμογή `GridSearchCV` με 5-fold stratified CV στο training set.

4. Επιλογή καλύτερων υπερπαραμέτρων βάσει CV accuracy.
5. Αξιολόγηση του tuned μοντέλου στο test set.

8.2 Πλέγματα υπερπαραμέτρων

Πίνακας 12: Υπερπαράμετροι που εξετάστηκαν στη διαδικασία Grid Search.

Μέθοδος	**Υποψήφιες τιμές**
Decision Tree	`criterion`: gini, entropy; `max_depth`: 1, 2, 3, 4, 5, 6, 8, 10, 12, None; `min_samples_leaf`: 1, 2, 5, 10
Bagging	`n_estimators`: 50, 100, 200; `max_samples`: 0.7, 1.0; `max_features`: 0.7, 1.0; `estimator__max_depth`: None, 6
Random Forest	`n_estimators`: 50, 100, 200; `max_features`: 3, 5, sqrt; `max_depth`: None, 6, 10; `min_samples_leaf`: 1, 2
Boosting	`n_estimators`: 50, 100, 200; `learning_rate`: 0.05, 0.1, 0.2; `max_depth`: 1, 2; `subsample`: 0.8, 1.0

8.3 Βέλτιστες υπερπαράμετροι και tuned test accuracy

Πίνακας 13: Βέλτιστες υπερπαράμετροι και ακρίβεια των tuned μοντέλων.

Μέθοδος	**Βέλτιστες υπερπαράμετροι**	**CV Acc.**	**Test Acc.**
Decision Tree	`criterion=gini`, `max_depth=6`, `min_samples_leaf=1`	0.9574	0.9654
Bagging	`estimator__max_depth=6`, `max_features=0.7`, `max_samples=1.0`, `n_estimators=50`	0.9679	0.9729
Random Forest	`max_depth=10`, `max_features=3`, `min_samples_leaf=1`, `n_estimators=100`	0.9690	0.9717
Boosting	`learning_rate=0.1`, `max_depth=2`, `n_estimators=200`, `subsample=1.0`	0.9665	0.9692

8.4 Σύγκριση untuned και tuned μοντέλων

Πίνακας 14: Σύγκριση αρχικών και βελτιστοποιημένων μοντέλων.

Μέθοδος	**Untuned Acc.**	**Tuned Acc.**	**Μεταβολή**	**Untuned Error**	**Tuned Error**
Decision Tree	0.9339	0.9654	+0.0315	0.0661	0.0346
Bagging	0.9660	0.9729	+0.0069	0.0340	0.0271
Random Forest	0.9666	0.9717	+0.0050	0.0334	0.0283
Boosting	0.9679	0.9692	+0.0013	0.0321	0.0308

8.5 Πίνακες σύγχυσης tuned μοντέλων

Πίνακας 15: Πίνακες σύγχυσης για τα βελτιστοποιημένα μοντέλα.

Μέθοδος	**TN**	**FP**	**FN**	**TP**
Decision Tree	367	24	31	1167
Bagging	370	21	22	1176
Random Forest	368	23	22	1176
Boosting	364	27	22	1176

8.6 Σχολιασμός της επίδρασης του tuning

Η διαδικασία tuning βελτίωσε την απόδοση όλων των μοντέλων. Η μεγαλύτερη βελτίωση παρατηρήθηκε στο απλό δέντρο ταξινόμησης. Το αρχικό δέντρο είχε `max_depth=2` και test accuracy 0.9339. Μετά το tuning, το βέλτιστο βάθος έγινε `max_depth=6`, και η test accuracy αυξήθηκε σε 0.9654. Η βελτίωση αυτή είναι σημαντική:

$$0.9654 - 0.9339 = 0.0315.$$

Αυτό δείχνει ότι το αρχικό δέντρο ήταν υπερβολικά απλό και δεν αξιοποιούσε πλήρως την πληροφορία των δεδομένων.

Στις ensemble μεθόδους, η βελτίωση ήταν μικρότερη αλλά υπαρκτή. Αυτό είναι αναμενόμενο, διότι τα αρχικά ensemble μοντέλα είχαν ήδη πολύ υψηλή ακρίβεια. Το Bagging βελτιώθηκε από 0.9660 σε 0.9729. Το Random Forest βελτιώθηκε από 0.9666 σε 0.9717. Το Boosting βελτιώθηκε από 0.9679 σε 0.9692.

Η μικρότερη βελτίωση εμφανίζεται στο Boosting. Αυτό δεν σημαίνει ότι το Boosting είναι αδύναμο μοντέλο· αντίθετα, το αρχικό Boosting ήταν ήδη το καλύτερο από τα untuned μοντέλα. Επομένως υπήρχε μικρότερο περιθώριο βελτίωσης μέσω tuning.

8.7 Καλύτερη μέθοδος ταξινόμησης

Αν ληφθούν υπόψη μόνο τα αρχικά μοντέλα των ερωτημάτων 1–4, η καλύτερη ακρίβεια προκύπτει από το Boosting:

Πίνακας 16: Κατάταξη αρχικών μοντέλων ως προς test accuracy.

Μέθοδος	**Untuned Test Accuracy**
Boosting	0.9679
Random Forest	0.9666
Bagging	0.9660
Decision Tree	0.9339

Μετά τη βελτιστοποίηση, η καλύτερη ακρίβεια προκύπτει από το Bagging:

Πίνακας 17: Κατάταξη tuned μοντέλων ως προς test accuracy.

Μέθοδος	**Tuned Test Accuracy**
Bagging	0.9729
Random Forest	0.9717
Boosting	0.9692
Decision Tree	0.9654

Συνολικά, η καλύτερη μέθοδος ταξινόμησης είναι το tuned Bagging, καθώς πετυχαίνει τη μεγαλύτερη ακρίβεια στο test set:

$$\text{Test Accuracy} = 0.9729$$

και το μικρότερο test error:

$$\text{Test Error} = 0.0271.$$

Η διαφορά του tuned Bagging από το tuned Random Forest είναι μικρή (0.9729 έναντι 0.9717), επομένως και οι δύο μέθοδοι έχουν πρακτικά πολύ καλή απόδοση. Ωστόσο, με βάση το κριτήριο που ζητείται στην εργασία, δηλαδή την test accuracy, το Bagging είναι το καλύτερο μοντέλο.

9 Συνολικά συμπεράσματα

Η πρόβλεψη του τύπου κρασιού με βάση τα φυσικοχημικά χαρακτηριστικά είναι ένα πρόβλημα στο οποίο οι μέθοδοι δέντρων και ensemble ταξινομητών αποδίδουν πολύ καλά. Ακόμη και το απλό δέντρο βάθους 2 πετυχαίνει accuracy 0.9339, γεγονός που δείχνει ότι οι δύο κατηγορίες κρασιού διαχωρίζονται αρκετά καλά από ορισμένες βασικές μεταβλητές, όπως τα θειώδη και τα χλωρίδια.

Οι ensemble μέθοδοι βελτιώνουν αισθητά την απόδοση. Το Bagging και το Random Forest μειώνουν τη διακύμανση συνδυάζοντας πολλά δέντρα, ενώ το Boosting χτίζει διαδοχικά ένα ισχυρό μοντέλο από απλούς ασθενείς ταξινομητές. Στα αρχικά μοντέλα, το Boosting είχε την καλύτερη ακρίβεια, ίση με 0.9679.

Η μελέτη του βάθους του δέντρου έδειξε ότι η αύξηση της πολυπλοκότητας βελτιώνει το μοντέλο μέχρι ένα σημείο. Το καλύτερο βάθος ήταν 6, ενώ για μεγαλύτερα βάθη το test error άρχισε να αυξάνεται, ένδειξη υπερπροσαρμογής. Αντίστοιχα, η μελέτη του `n_estimators` έδειξε ότι Bagging και Random Forest σταθεροποιούνται γρήγορα, ενώ το Boosting χρειάζεται περισσότερους εκτιμητές για να φτάσει στη βέλτιστη απόδοση.

Τέλος, το hyperparameter tuning βελτίωσε όλα τα μοντέλα. Η μεγαλύτερη βελτίωση εμφανίστηκε στο απλό δέντρο, ενώ στις ensemble μεθόδους οι βελτιώσεις ήταν μικρότερες επειδή οι αρχικές επιδόσεις ήταν ήδη υψηλές. Το καλύτερο τελικό μοντέλο ήταν το tuned Bagging, με test accuracy 0.9729.

Α΄ Αρχεία υλοποίησης

Η ανάλυση υλοποιήθηκε με τα εξής scripts:

- `q1_decision_tree.py`
- `q2_bagging.py`
- `q3_random_forest.py`
- `q4_boosting.py`
- `q5_tree_depth_error.py`
- `q6_ensemble_n_estimators_error.py`
- `q7_hyperparameter_tuning.py`

Τα βασικά παραγόμενα αρχεία αποτελεσμάτων είναι:

- `q5_tree_depth_error.png`
- `q6_ensemble_n_estimators_error.png`
- `q7_hyperparameter_tuning_summary.csv`
- `q7_final_comparison.md`